Learning-Aware Fault Tolerance in Distributed Reinforcement Learning Systems

Shravani Pranay Sawant
Stevens Institute of Technology
ssawant10@stevens.edu

Rui Wei
Stevens Institute of Technology
rwei7@stevens.edu

Hao Wang
Stevens Institute of Technology
hwang9@stevens.edu

***Abstract*—Distributed reinforcement learning (RL) systems are increasingly deployed in large-scale and dynamic environments where failures and resource variability are unavoidable. While fault tolerance has been extensively studied in distributed systems, its implications in RL remain less understood due to the tight coupling between system execution and learning dynamics. In contrast to traditional workloads, disruptions in RL systems may affect not only system performance but also the progression of learning.**

In this work, we present an empirical study of fault tolerance in distributed RL systems from a learning-aware perspective. We first analyze the impact of partial worker failures by simulating controlled scaling-down scenarios using a logical scaling framework, enabling isolation of failure effects without system reconstruction. Our experiments across custom environments and standard benchmarks, including MuJoCo and Atari tasks, show that RL training often continues under partial worker failures, although the degree of performance degradation depends on the environment, scaling point, and training configuration.

We further investigate the cost of recovery through checkpoint-based mechanisms and observe that checkpointing overhead is relatively small compared to other components of the training pipeline. A detailed latency analysis reveals that model updates and worker-related operations dominate runtime, while checkpointing contributes minimally under the evaluated configurations.

Finally, we provide preliminary evidence that actor contributions during training are not uniform, suggesting that treating all failures equally may be inefficient. These findings motivate the need for learning-aware fault tolerance strategies that prioritize components based on their impact of learning rather than applying uniform recovery mechanisms.

Index Terms: Reinforcement Learning, Distributed Systems, Fault Tolerance, RLlib, Checkpointing, Learning-Aware Systems

I. Introduction

Reinforcement learning (RL) has become a widely adopted paradigm for sequential decision-making, enabling agents to learn complex behaviors through interaction with dynamic environments [1]. Advances in deep RL have significantly expanded its applicability across domains such as continuous control and high-dimensional perception tasks [2]–[4]. However, these improvements come with substantial computational requirements, often necessitating large-scale data collection and long training durations.

To address these challenges, modern RL systems employ distributed architectures in which multiple workers interact with the environment in parallel while a centralized learner updates the policy [6]. Early systems such as IMPALA demonstrate that decoupling data collection from learning can significantly improve throughput and scalability [6]. More recent work, including serverless RL frameworks such as MinionsRL [7], Stellaris [8], and Nitro [9], further extend this paradigm by enabling dynamic scaling of workers to optimize resource utilization and reduce operational cost.

Despite these advances, distributed RL systems operate in environments where failures and resource fluctuations are common. Large-scale distributed systems are inherently failure-prone, with prior studies showing that failures occur frequently in production machine learning workloads and can significantly impact system behavior [21]. As a result, fault tolerance mechanisms such as checkpointing and rollback-based recovery have become essential components of distributed training systems [16], [17].

However, the role of fault tolerance in RL systems differs fundamentally from that in traditional distributed workloads. Unlike supervised learning, which relies on static and independent datasets, RL generates data online through interactions with the environment. This results in a non-stationary and policy dependent data distribution, where disruptions in system execution can influence both efficiency and learning progression. Existing fault tolerance approaches, including checkpointing-based recovery [16], [17] and retry strategies [20], are largely system-centric and focus on ensuring correctness, without explicitly considering the impact of failures on learning dynamics.

In this work, we adopt a learning-aware perspective to study fault tolerance in distributed RL systems. Instead of directly proposing a new recovery mechanism, we first aim to understand how failures affect both system performance and learning behavior. We structure our study around three key questions: (i) how partial worker failures affect learning stability and system performance, (ii) what the cost of checkpointing-based recovery is in practical RL settings, and (iii) whether different actors contribute equally to learning.

To investigate these questions, we implement a logical scaling framework using RLlib [18], where the number of active workers is reduced during training without rebuilding the system. This approach allows us to simulate failure scenarios in a controlled manner while preserving the structure of the training pipeline. We conduct experiments across both custom

environments and standard benchmarks, including MuJoCo and Atari tasks, and analyze training behavior, system latency, and checkpoint overhead.

Our results show that RL training can tolerate partial worker failures in many cases, but the impact is not uniform; some environments exhibit moderate reward degradation depending on when scaling occurs. Furthermore, we find that checkpointing overhead is relatively small compared to other components of the training, suggesting that recovery cost is not the dominant bottleneck. Finally, we observe variability in actor contributions during training, which motivates the need for fault tolerance strategies that consider the learning impact of system components rather than treating all failures uniformly.

The main contributions of this work are summarized as follows:

- We present a systematic study of failure impact in distributed RL using a logical scaling framework that simulates partial worker failures without system reconstruction.
- We provide a detailed analysis of checkpoint overhead and show that it is not the dominant cost in the RL training under the evaluated settings.
- We offer preliminary insights into actor importance, highlighting variability in learning contributions across workers and motivating learning aware fault tolerance strategies.

The remainder of this paper is organized as follows. Section II introduces the problem and system model. Section III describes the methodology and experimental setup. Section IV presents the experimental results. Section V discusses implications and limitations. Section VI reviews related work, and Section VII concludes the paper.

II. Problem Statement

Modern distributed reinforcement learning (RL) systems rely on parallel data collection and centralized learning to achieve scalability and efficiency. In a typical architecture, multiple workers (actors) interact with independent instances of the environment to generate trajectories, while a centralized learner updates the policy using the collected experience [6], [15]. This actor-learner paradigm enables high-throughput training but introduces tight coupling between system execution and learning dynamics.

Unlike supervised learning, where training data is static and independent, RL operates on data generated online by the current policy. This results in a non-stationary and dependent data distribution, making the training process inherently sensitive to system-level variation [1], [6]. In large-scale deployments, such systems often operate under dynamic conditions where worker availability may change due to failures, preemption, or resource scaling [21]. As a result, understanding the impact of such changes is critical for designing effective fault tolerance mechanisms.

In this work, we consider a distributed RL system based on the actor-learner architecture, as illustrated in Fig. 1. The system consists of an initial set of N workers that interact with the environment in parallel to collect trajectories. These trajectories are transmitted to the learner, which performs policy updates and synchronizes updated parameters back to the workers. This iterative process forms the core training loop of distributed RL systems.

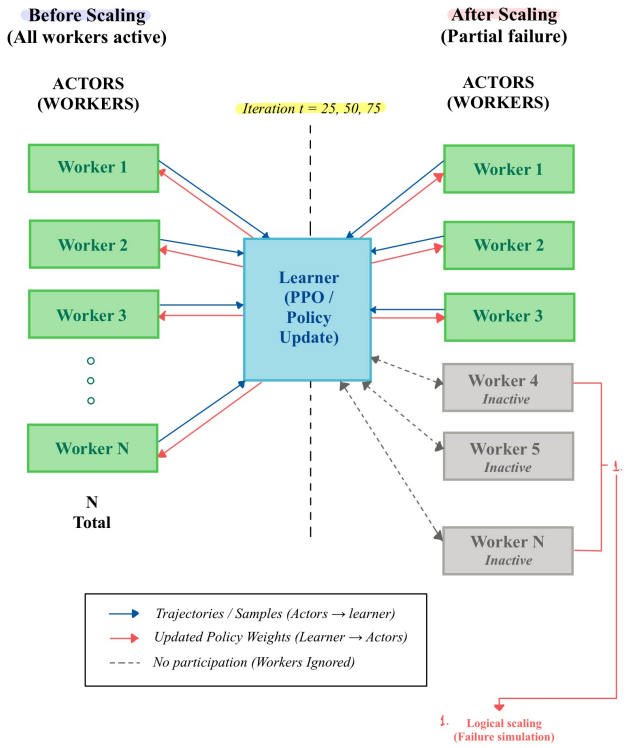


Fig. 1: Distributed RL system model with logical scaling. The system transitions from full worker participation to a reduced active set during training to simulate partial failures

To study the impact of failures in a controlled manner, we model partial worker failures as a reduction in the number of active workers during training. Instead of physically removing workers or reconstructing the system, we implement a logical scaling mechanism in which only a subset of workers actively participate in data collection while the remaining workers are ignored. This approach preserves the system structure and avoids introducing additional overhead from system reinitialization or checkpoint-based recovery.

More formally, the system transitions from an initial configuration with N active workers to a reduced configuration with $N' < N$ active workers at a predefined iteration. Active workers continue to generate trajectories and contribute to learning, while inactive workers remain idle. This setup allows us to isolate the effect of reduced worker participation on both system performance and learning behavior.

Existing fault tolerance mechanisms in distributed machine learning systems primarily focus on ensuring correctness

through checkpointing and rollback strategies [16], [17]. While such approaches are effective in recovering from failures, they do not explicitly account for the characteristics of RL workloads. In particular, it remains unclear how different components of the RL training loop, such as data sampling, model updates, synchronization, and auxiliary operations, contribute to overall system latency under failure conditions.

Based on these observations, we formulate the following research problem:

> In a distributed reinforcement learning system with dynamic worker participation, how do partial worker failures affect learning behavior and system performance, and how should fault tolerance mechanisms be designed to account for their impact on learning?

To address this problem, we focus on three key objectives:

1) to analyze the impact of partial worker failures on learning stability and convergence,
2) to quantify the cost of checkpoint-based recovery and its contribution to overall system latency, and
3) to examine whether different actors contribute equally to learning and how this influences fault tolerance design.

These objectives guide the experimental methodology and analysis presented in the subsequent sections.

III. Learning-Aware Fault Tolerance Framework

In this section, we present the design and experimental framework used to study fault tolerance in distributed reinforcement learning systems. Our goal is not to introduce a new recovery mechanism, but to build a structured and measurable framework that enables systematic analysis of failure impact, recovery cost and variability in actor contributions. The framework is designed to preserve the integrity of RL training while allowing controlled manipulation of system behavior.

A. Design Motivation

Distributed RL systems operate under a fundamentally different paradigm compared to traditional machine learning workloads. In RL, data is generated online through interactions between the agent and the environment, and the quality of this data directly influences policy updates. As a result, system-level disruptions such as worker failures or resource fluctuations can impact not only execution efficiency but also learning dynamics.

Existing fault tolerance approaches in distributed systems focus on correctness and recovery, primarily through checkpointing and restart mechanisms [16], [17]. While these methods ensure continuity of execution, they do not provide insights into how failures affect learning behavior or whether all system components contribute equally to training. This limitation motivates the need for a framework that enables controlled experimentation and deeper analysis of distributed RL systems under failure conditions.

To address this gap, we embrace a learning-aware perspective, focusing on understanding how failures interact with both system performance and learning outcomes. Rather than injecting failures through system crashes, which introduces additional overhead, we simulate failures through logical scaling, enabling clean and reproducible experiments.

B. Logical Scaling Framework

To model partial worker failures, we introduce a logical scaling mechanism that dynamically reduces the number of active workers during training. Unlike conventional scaling approaches that physically remove workers, our framework maintains a fixed set of workers and selectively controls their participation in sampling.

Initially, all N workers are active and contribute to trajectory generation. At a predefined iteration t , the system transitions to a reduced configuration with $N' < N$ active workers. The remaining workers are not removed from the system but are simply ignored during data collection and learning.

This design allows us to simulate failure scenarios without introducing additional overhead from system reinitialization or checkpoint recovery. From a fault tolerance perspective, inactive workers can be interpreted as failed or unavailable nodes, enabling controlled analysis of failure impact.

The key advantage of this approach is that it isolates the effect of reduced worker participation while preserving the overall system structure. As a result, any observed changes in system performance or learning behavior can be attributed directly to the scaling event.

C. Training Workflow with Logical Scaling

The complete training workflow, including logical scaling and measurement components, is illustrated in Fig. 2.

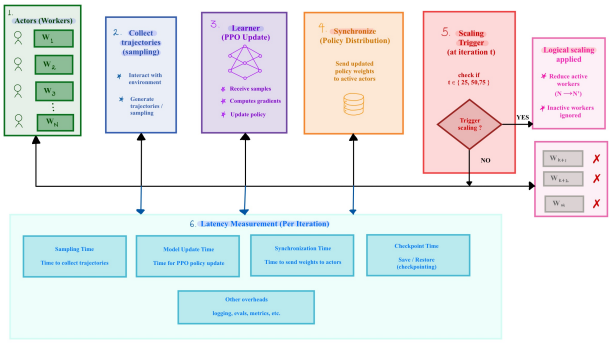


Fig. 2: Training workflow with logical scaling and latency measurement in a distributed RL system, illustrating actor-learner interaction, scaling trigger, and per-iteration latency components.

As shown in Fig. 2, the system follows a standard actor-learner pipeline. During each iteration, active workers interact with the environment to collect trajectories, which are then sent to the centralized learner. The learner performs policy updates using the PPO algorithm and synchronizes updated parameters back to the workers.

A key component of the workflow is the scaling trigger, which is evaluated at each iteration. Once the predefined

scaling condition is satisfied (e.g. after iteration 25; after iteration 50; after iteration 75), the system reduces the number of active workers. Importantly, this transition occurs without interrupting the training process, ensuring continuity in both system execution and learning.

In addition to the core training loop, the workflow integrates latency measurement at each stage of the pipeline. This includes sampling time model update time, synchronization overhead, and checkpoint-related operations. By embedding measurements directly into the training loop, we obtain a fine-grained view of system behavior under both static and scaled configurations. This workflow design enables systematic comparison of system performance before and after scaling, providing a foundation for analyzing the impact of failures and recovery mechanisms.

D. Measurement and Latency Breakdown

To understand system behavior in detail, we instrument the training process to capture fine-grained latency components at each iteration. Rather than treating iteration time as a single aggregated metric, we decompose it into multiple stages, including data sampling, model updates, synchronization, checkpoint operations, and auxiliary overheads.

This decomposition allows us to identify which components dominate execution time, and how their contributions change under scaling conditions. In particular, it enables us to evaluate whether checkpointing introduces significant overhead compared to other parts of the training pipeline.

By analyzing latency at this level of granularity, we gain insights into the relative cost of different system operations. These insights are critical for understanding whether traditional fault tolerance mechanisms are sufficient or whether alternative strategies are needed.

E. Learning-Aware Fault Tolerance Perspective

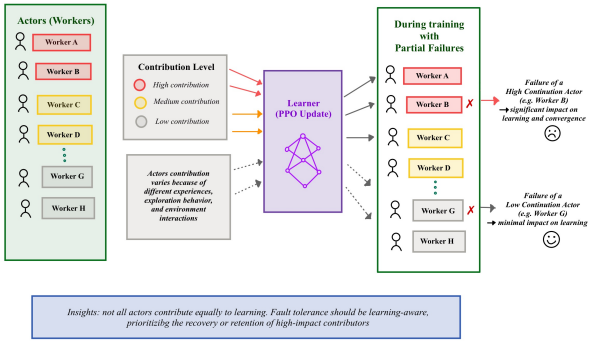


Fig. 3: Conceptual illustration of learning-aware fault tolerance. Actor contributions vary across workers, suggesting that failures may have an unequal impact on learning.

Beyond system-level performance, we examine the variability in actor contributions during training. In distributed RL systems, different workers may generate trajectories with varying levels of usefulness due to differences in exploration behavior, stochasticity, and environment interactions [14], [15].

This variability suggests that not all actors contribute equally to learning. As illustrated in Fig. 3, failures affecting high-contribution actors may have a more significant impact on training compared to failures affecting low-contribution actors.

Fig. 3 provides a conceptual illustration of this idea. Actors are categorized based on their relative contribution to learning, and impact of failures varies accordingly. This observation highlights a key limitation of uniform fault tolerance strategies, which treat all components equally regardless of their contribution.

While our framework does not explicitly implement a learning-aware recovery mechanism, these observations motivate the need of such approaches. In particular, they suggest that future fault tolerance strategies in RL systems should prioritize components based on their impact on learning rather than applying uniform protection.

Overall, the proposed framework enables systematic analysis of distributed RL systems under failure conditions by combining logical scaling, detailed measurement, and multi-environment evaluation. It provides a foundation for understanding both system performance and learning behavior, which are together important for designing effective fault tolerance mechanisms.

IV. Numerical Results and Analysis

This section presents a detailed empirical evaluation of the proposed framework from three complementary perspectives:

- RQ1: The impact of failures on reinforcement learning behavior,
- RQ2: The cost of recovery mechanisms, and
- RQ3: The role of actor contributions in shaping fault tolerance strategies.

All results are obtained through controlled experiments on distributed PPO training using RLlib across both continuous control (MuJoCo) and discrete (Atari) environments.

A. Experimental Setup

All experiments are conducted using a distributed actor-learner architecture, where multiple workers interact with independent environment instances and periodically synchronize with a centralized learner. We evaluate five benchmark environments: Hopper, Humanoid, Walker2d (MuJoCo), and SpaceInvaders, Qbert (Atari). Each configuration is executed across 10 random seeds to ensure statistical consistency, and results are reported using mean trends along with variability bands.

To simulate failures, we employ the logical scaling mechanism described in Section III. Starting from an initial configuration of N workers, the system transitions to a reduced active set $N \ / \ 2$ at predefined iterations (25, 50, and 75). Importantly, inactive workers remain part of the system but do not participate in sampling, allowing us to isolate the effect of reduced participation without introducing reinitialization overhead.

TABLE I: Impact of Partial Worker Failures on Learning Performance

Env.	Scale	Impact	Convergence	Observation
Hopper	25, 50, 75	Minimal	Stable	Moderate final-reward change in some scaling settings.
Humanoid	25, 50, 75	Slight	Smooth progression	Generally stable with slight degradation.
Walker2d	25, 50, 75	Minimal	Stable	Mostly stable, with occasional reward variation.
SpaceInvaders	25, 50, 75	Variable	Noisy but stable	High variance; scaling impact is more environment-dependent.
Qbert	25, 50, 75	Variable	Fluctuating	Sensitive; reward variation is more pronounced.

For system-level analysis, we instrument the training loop to record fine-grained latency components as we had described in the prior section. Additionally, experiments are repeated across different model sizes and checkpoint frequencies to evaluate scalability and recovery cost.

B. Impact of Failures on Learning Behavior (RQ1)

We begin by examining whether partial failures disrupt reinforcement learning itself. Fig. 4 shows the baseline learning curve for the Hopper environment under static conditions. The agent exhibits stable improvement in episodic return over time, with gradual convergence and moderate variance across seeds. This serves as a reference for evaluating the effect of scaling.

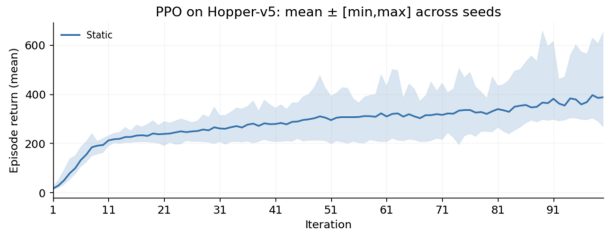


Fig. 4: Baseline training behavior under static configuration. The agent demonstrates consistent learning progress with moderate variability, providing a reference point for analyzing the effects of scaling and partial failures.

Fig. 5 presents the corresponding results when scaling is introduced at different stages of training (iteration 25, 50, and 75). Across all three scenarios, a consistent pattern emerges. The learning curve generally continues after the reduction in active workers, but the final reward can be lower in some scaling settings. This indicates that scaling does not necessarily collapse training, but it can introduce moderate performance degradation depending on when it is applied. In particular, early scaling leads to slightly slower growth in reward, whereas later scaling has almost negligible impact on the final performance.

To validate whether this behavior generalizes beyond a single environment, Fig. 6 extends the analysis to additional benchmarks. Across Humanoid and Walker2d, which represent continuous control tasks, scaling introduces only marginal differences in convergence rate.

In contrast, Atari environments such as SpaceInvaders and Qbert exhibit higher variability, but the overall learning trend remains preserved. In these noisier settings, scaling does not always cause collapse, but it can produce larger reward variation and moderate degradation compared with continuous-control environments.

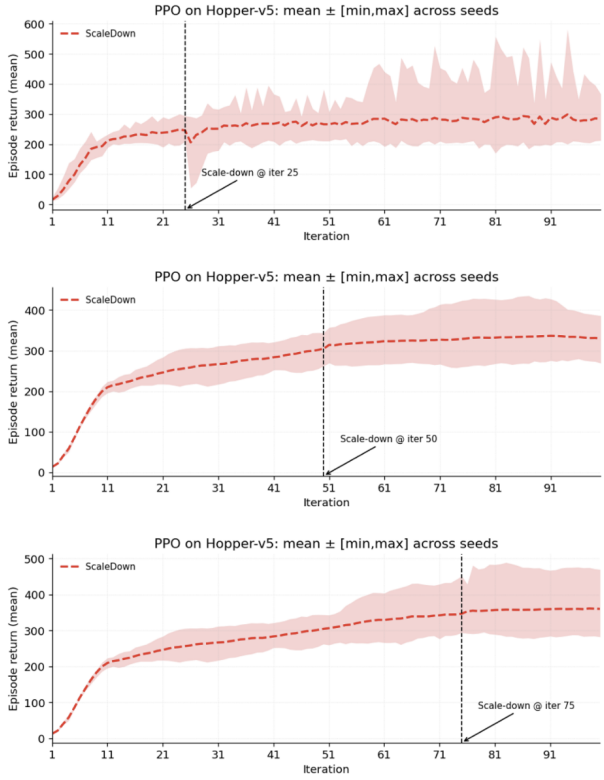
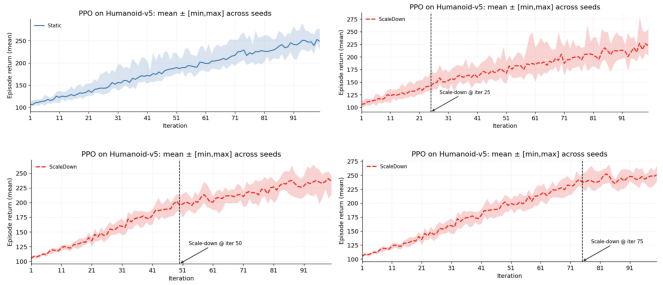


Fig. 5: Learning behavior under scaling at different training stages. Despite scale-down events at iterations 25, 50, and 75, the agent maintains stable learning trends, indicating resilience to partial worker reduction.

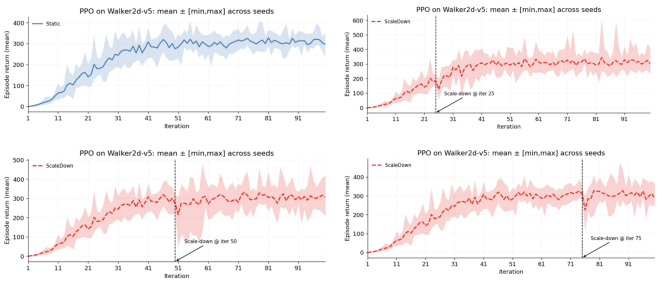
These observations suggest that distributed RL can tolerate partial worker failures in many cases, but the effect is not uniform across environments or scaling points. The impact of failures is not uniform; rather, it depends on when the failure occurs and the characteristics of the environment. Table I summarizes this behavior across environments, highlighting that while convergence speed may vary slightly, training often continues, while final reward and convergence behavior may change depending on the workload.

C. Efficiency Analysis under Scaling

While the previous subsection focuses on learning behavior, we now examine how scaling affects system efficiency. Fig. 7 compares reward progression as a function of wall-clock time



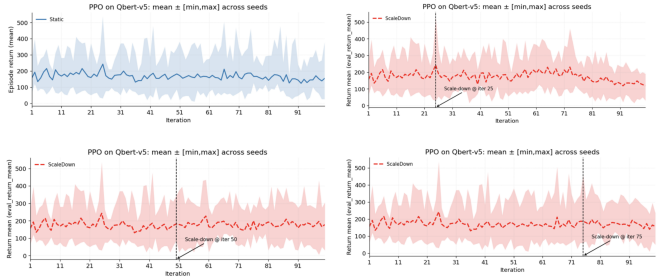
(a) Humanoid



(b) Walker2d



(c) SpaceInvaders



(d) Qbert

Fig. 6: Learning behavior under scaling across different environments. (A) Humanoid, (B) Walker2d, (C) SpaceInvaders, and (D) Qbert. Across environments, scaling events show environment-dependent effects: continuous-control tasks generally remain stable, while Atari tasks show higher variance and more noticeable reward variation.

for both static and scaled configurations. Unlike iteration-based plots, this view captures the actual runtime implications of scaling.

Across all environments, scaling leads to a noticeable reduction in wall-clock time per iteration. This is particularly evident in Hopper and Walker2d, where the scaled configuration reaches comparable reward levels faster than the static baselines. The reduction in active workers decreases coordination and sampling overhead, resulting in shorter iteration durations.

However, this improvement is not uniform across environments. In more complex settings such as SpaceInvaders, the benefit is less pronounced due to higher inherent variance in the learning process. Nevertheless, the key observation remains that scaling can improve computational efficiency, although its effect on learning outcomes varies across environments.

This result reinforces an important insight: reducing worker participation can be beneficial from a system perspective, provided that it does not disrupt the learning signal. The tradeoff between resource usage and training speed becomes a central consideration in designing scalable RL systems.

D. Recovery Cost and Checkpoint Overhead (RQ2)

We next evaluate the cost of recovery mechanisms, focusing on checkpointing as the primary fault tolerance strategy. Fig. 8 presents a detailed latency breakdown across different model sizes, comparing configurations of with and without failure.

Across all environments, model update time and sampling remains the dominant contributors to iteration latency. These components consistently account for the majority of execution time, regardless of model size or scaling condition. In contrast, checkpoint-related operations contribute a relatively small fraction of the total latency, especially at lower frequencies.

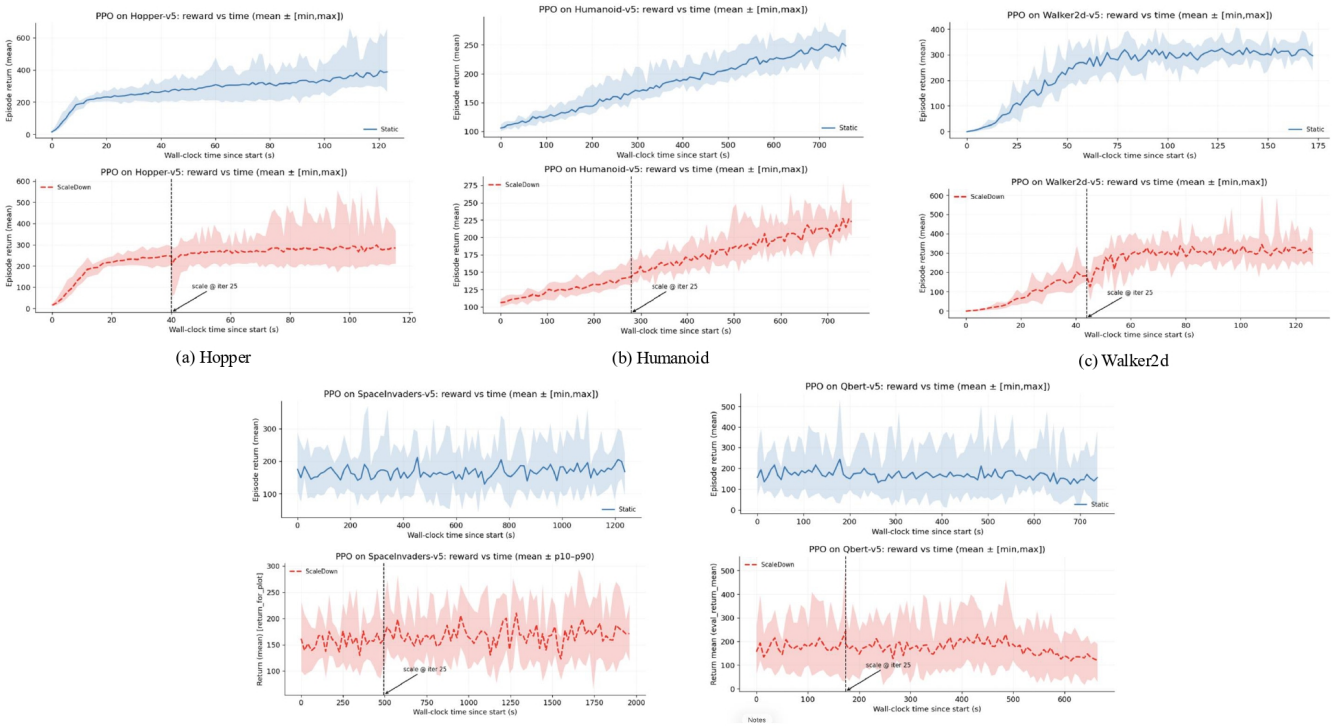
To further analyze this behavior, Fig. 9 examines the effect of checkpoint frequency across environments. As expected, increasing the frequency of checkpointing leads to higher overhead. At high frequencies (e.g. checkpointing every iteration), the cost becomes more visible, particularly in computationally intensive environments such as SpaceInvaders. However, even in these cases, checkpoint overhead does not dominate the total runtime.

Table II summarizes the relative contribution of checkpointing across configurations. The results indicate that while frequent checkpointing introduces measurable overhead, it is not the primary bottleneck in distributed RL training. Instead, the cost of core learning operations remains significantly higher.

These findings challenge a common assumption that checkpointing is the main source of inefficiency in fault tolerant systems. In the context of RL, recovery mechanisms appear to be relatively inexpensive compared to the overall cost of training.

E. Actor Contribution and Learning-Aware Fault Tolerance (RQ3)

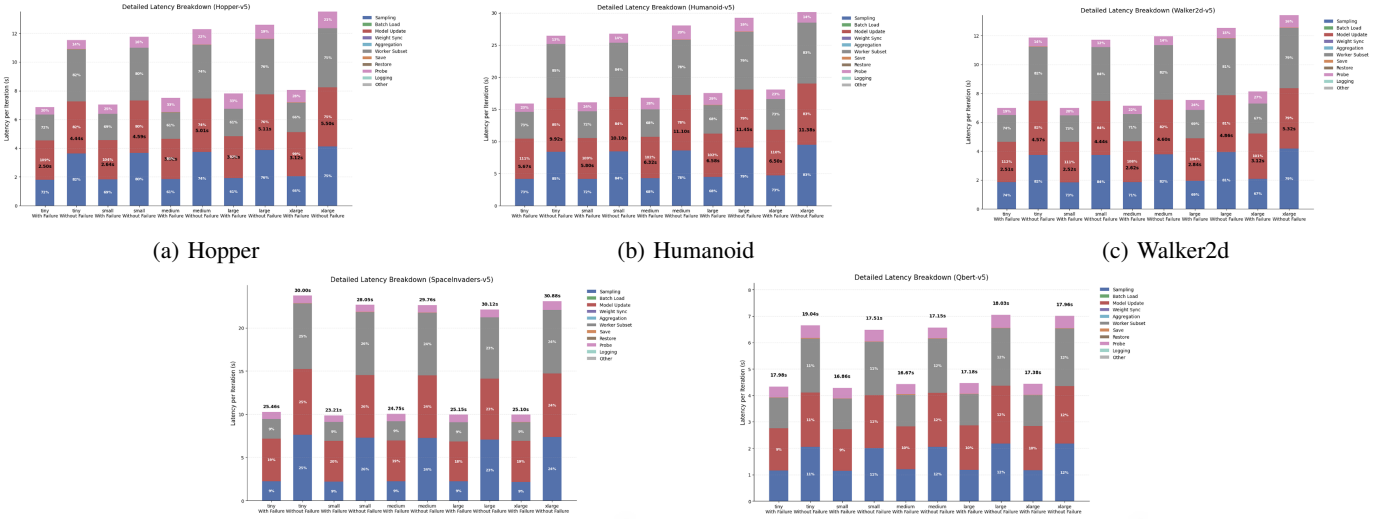
The final aspect of our analysis focuses on the role of individual actors in the learning process. Fig. 10 illustrates the distribution of actor importance scores across different stages of training. These scores are computed based on reward



(a) Hopper (b) Humanoid (c) Walker2d

(d) SpaceInvaders (e) Qbert

Fig. 7: Wall-clock performance comparison between static and scale-down training. Each subfigure shows episode return over time with scaling at iteration 25. Learning trends generally remain consistent across environments, with Hopper (A) and Walker2d (C) showing comparable growth, Humanoid (B) remaining stable, and stochastic environments (D–E) exhibiting higher variance. Overall, partial worker reduction improves system efficiency, with its impact on learning depending on the environment.



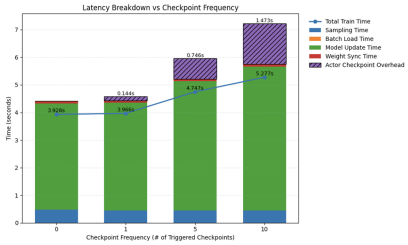
(a) Hopper (b) Humanoid (c) Walker2d

(d) SpaceInvaders (e) Qbert

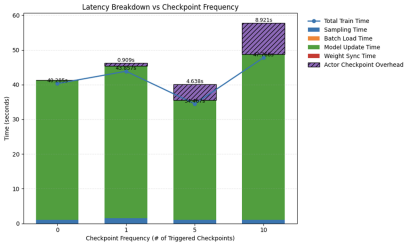
Fig. 8: Detailed latency breakdown across environments and model sizes under failure and non-failure settings. Model update and sampling dominate total latency, while checkpoint-related overhead remains comparatively small.

statistics and trajectory characteristics, capturing the relative contribution of each worker.

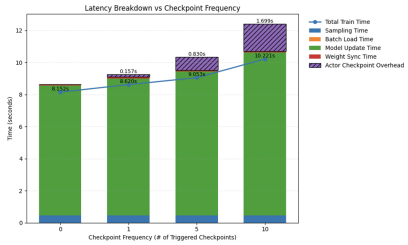
Across all environments, the distribution of importance is highly skewed. A subset of actors consistently exhibits higher contribution, while others have relatively limited impact. This imbalance persists across early, mid, and late stages of training,



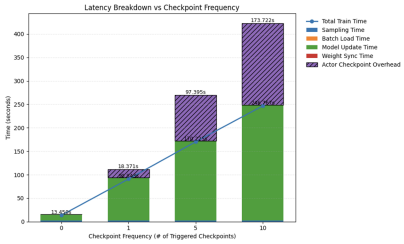
(a) Hopper



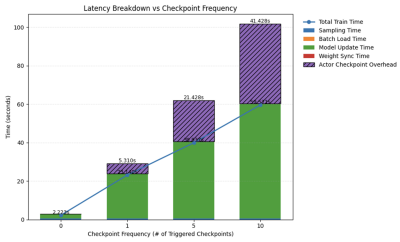
(b) Humanoid



(c) Walker2d



(d) SpaceInvaders

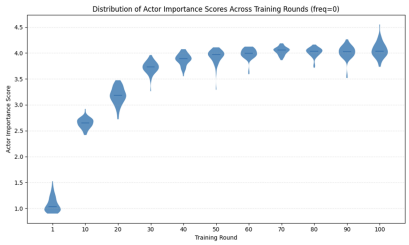


(e) Qbert

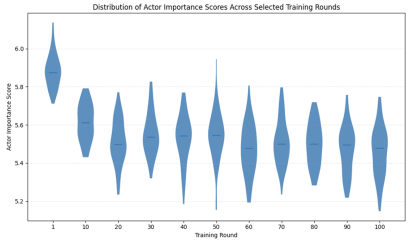
Fig. 9: Impact of checkpoint frequency on latency across environments. Increasing checkpoint frequency leads to higher overhead, particularly in computationally intensive environments such as SpaceInvaders, while core learning components remain dominant.

TABLE II: Checkpoint overhead across environments with 50 runners. Higher checkpoint frequency increases overhead, with Atari environments showing larger relative checkpoint cost than MuJoCo environments.

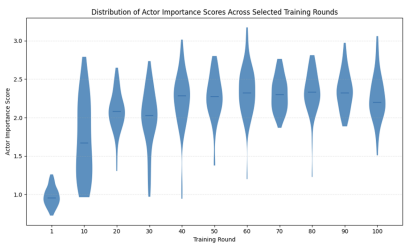
Environment	Freq = 0 Total (s)	Freq = 1 Ckpt (s)	Freq = 5 Ckpt (s)	Freq = 10 Ckpt (s)	Observation
Hopper	3.928	0.144	0.746	1.473	Overhead grows gradually
Humanoid	40.285	0.909	4.638	8.921	Training cost dominates
Walker2d	8.152	0.157	0.830	1.699	Low-to-moderate overhead
SpaceInvaders	13.450	18.371	97.395	173.722	Checkpoint cost becomes large
Qbert	2.223	5.310	21.428	41.428	High relative checkpoint cost



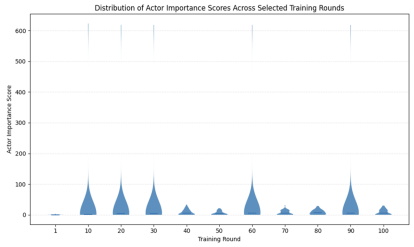
(a) Hopper



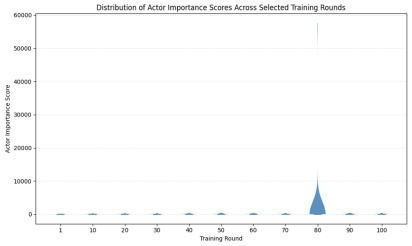
(b) Humanoid



(c) Walker2d



(d) SpaceInvaders



(e) Qbert

Fig. 10: Distribution of actor importance scores across training rounds for different environments (50 actors). Actor contributions are uneven, with a subset of actors consistently showing higher influence, especially in complex environments where variability is more pronounced.

although the exact distribution evolves over time.

Notably, environments with higher stochasticity, such as Atari tasks, show greater variability in importance scores, reflecting the diversity of exploration patterns. In contrast, con-

TABLE III: Summary of actor importance statistics across environments (50 actors). Actor contributions are non-uniform, with higher variance and stronger dominance of top actors in complex environments.

Environment	Mean Score	Variance Level	Max Score (approx.)	Top Actor Contribution (%)	Insight
Hopper	∼4.0	Low	∼4.5	∼15–20%	Stable and near-uniform contributions
Humanoid	∼5.5	Moderate	∼6.1	∼20–25%	Consistent with moderate variability
Walker2d	∼2.2	Moderate	∼3.1	∼20–25%	Gradual stabilization over time
SpaceInvaders	–	High	∼600+	∼30–40%	High variance with occasional extreme spikes
Qbert	–	Very High	∼57k+	∼40–50%	Strongly skewed; dominated by few actors

tinuous control environments exhibit more stable distributions, indicating more consistent actor behavior.

Table III provides a summary of these statistics, including mean importance, variance, and the contribution of top-performing actors. The results confirm that learning is not evenly distributed across workers. Instead, a small number of actors often account for a significant portion of the useful training signal.

This observation has direct implications for fault tolerance. If all actors are treated equally, the system may expend resources protecting components that contribute little to learning. A more effective approach would prioritize high-impact actors, enabling selective recovery or protection strategies. This forms the basis of learning-aware fault tolerance, where system decisions are guided by their influence on training outcomes rather than purely structural considerations.

F. Summary of Findings

The experimental results presented in this section lead to three key insights. First, partial failures in distributed RL do not always collapse learning; their impact ranges from minor disturbance to moderate reward degradation depending on timing, environment characteristics, and training configuration. Second, checkpointing overhead is relatively small compared to core training operations, suggesting that recovery cost is not the dominant challenge. Third, actor contributions are highly uneven, indicating that fault tolerance strategies should account for the varying importance of system components.

Together, these findings motivate a shift from uniform fault tolerance mechanisms toward approaches that are aware of learning dynamics. By aligning system-level decisions with their impact on training, it becomes possible to design more efficient and effective distributed RL systems.

V. Discussion Implications

The results presented in Section IV provide a consistent picture of how distributed RL systems behave under partial failures and recovery mechanisms. Beyond the numerical observations, these findings have broader implications for how fault tolerance should be designed in RL-based systems.

A. Rethinking failures in reinforcement learning systems

A key takeaway from our experiments is that partial worker failures do not always lead to catastrophic breakdown of learning. Across multiple environments and scaling points, the learning process often continues, but the degree of reward degradation and convergence change varies across environments. This stands in contrast to traditional assumptions in distributed systems, where failures are often treated as events that must be immediately corrected to preserve correctness and performance.

In reinforcement learning, however, the situation is fundamentally different. Training relies on continuously generated data rather than a fixed dataset, and the learning signal is inherently stochastic. As a result, the system can tolerate reduced sampling capacity in some settings, but this tolerance is not uniform. Similar observations about the robustness of learning-based systems have been noted in empirical studies of deep learning workloads, where failures are frequent but do not always result in training collapse [21].

This suggests that failures in RL systems should not be viewed purely as disruptions to be eliminated, but rather as events whose impact depends on their interaction with the learning process. In particular the timing of failures and the nature of the environment play a significant role in determining their effect.

B. Revisiting the role of checkpointing

Checkpointing has long been considered a central mechanism for fault tolerance in distributed machine learning systems. Prior work has focused on improving the efficiency of checkpointing and recovery, proposing techniques to reduce overhead and speed up restoration [16], [17], [20]. While these approaches are valuable, our results indicate that checkpointing may not be the dominant factor in overall system performance for RL workloads.

Across all evaluated environments, the cost of checkpointing remains relatively small compared to core training operations such as sampling and model updates. Even at higher checkpoint frequencies, the overhead increases gradually and does not dominate iteration time. This observation aligns with system-level studies showing that computation and communication costs often outweigh recovery mechanisms in large-scale training systems [18], [19].

The implication is that optimizing checkpointing alone may not significantly improve the efficiency of RL training. Instead, greater gains may be achieved by focusing on the components that contribute most to runtime, particularly those related to data generation and policy updates.

C. Non-Uniform Contribution of Actors

Perhaps the most important insight from this work is that not all actors contribute equally to the learning process. The analysis of actor importance reveals a highly skewed distribution, where a subset of workers consistently provides more valuable training data.

This observation is consistent with ideas from reinforcement learning literature that emphasize the importance of data quality and prioritization. For example, prioritized experience replay highlights that certain experiences are more informative for learning than others [14], [15]. Similarly, work in federated learning has shown that selecting more informative clients can significantly improve training efficiency.

In our settings, this imbalance manifests at the level of actors. Differences in exploration behavior, interaction trajectories, and environmental stochasticity lead to variation in the usefulness of collected data. As a result, treating all actors uniformly may lead to inefficient use of resources, particularly in the presence of failures.

D. Toward learning-aware fault tolerance

Combining the above insights leads to a broader perspective on fault tolerance in distributed RL systems. Traditional approaches are largely system-centric, focusing on ensuring that all components are preserved or restored uniformly. However, our results suggest that such uniform treatment may not be optimal.

Instead, we argue for a shift toward learning-aware fault tolerance, where system-level decisions are guided by their impact on training outcomes. In this paradigm:

- failures are evaluated based on their effect on learning rather than their structural role,
- recovery mechanisms prioritize components that contribute more significantly to training,
- resource allocation is informed by the relative importance of actors.

This perspective opens the door to more adaptive strategies, such as selective checkpointing, importance-based actor scheduling, and dynamic scaling policies that consider both system efficiency and learning effectiveness.

E. Limitations and Future Directions

While we studied a lot through the current findings, several limitations should be acknowledged. First, the experiments are conducted using PPO within RLlib, and the conclusions may not directly generalize to all RL algorithms. Second, the evaluation focuses on a specific set of environments, which, although diverse, do not cover the full range of RL applications. Third, the actor importance metric is based on reward statistics and does not capture all aspects of contribution, such as long-term exploration value.

In future work, we can extend this analysis by incorporating additional algorithms, exploring more complex environments, and developing more refined measures of actor contribution. Another important direction is to design and evaluate concrete learning-aware fault tolerance mechanisms that leverage these insights in practice.

VI. Related Work

The proposed work intersects multiple areas, including distributed reinforcement learning, fault tolerance in machine learning systems, and learning-aware data prioritization. In this section, we position our contributions within these domains.

A. DRL frameworks

Modern distributed RL systems are built on actor-learner architectures that separate data collection from policy updates. Frameworks such as IMPALA [6] and distributed prioritized experience replay [15] have demonstrated the effectiveness of this design in scaling RL training. More recent systems, including serverless approaches such as MinionRL [7], Stellaris [8], and Nitro [9], focus on improving scalability, resource efficiency, and elasticity in distributed environments.

While these systems explore dynamic scaling and resource management, they primarily target throughput and cost optimization. They do not explicitly study how failures or changes in worker participation affect the learning process itself. Our work builds on these architectures but shifts the focus towards understanding the interaction between system dynamics and learning behavior.

B. Fault Tolerance in Distributed Machine Learning

Fault tolerance has been extensively studied in distributed system and machine learning frameworks. Traditional approaches rely on checkpointing and rollback mechanisms to ensure correctness in the presence of failures [16], [18], [19]. Recent work has proposed optimized recovery strategies, such as in-memory checkpointing (GEMINI) [16] and expedited recovery techniques (Swift) [17], as well as adaptive retry mechanisms (Sibylla) [20].

These works primarily aim to reduce recovery time and system overhead. However, they treat all components of the system uniformly and do not consider the effect of failures on the learning process. In contrast, our work examines not only the cost of recovery but also the impact of failures on training behavior, providing a more holistic view of fault tolerance in RL systems.

C. Learning-aware prioritization and data selection

The idea that not all data contributes equally to learning is well established in machine learning. Prioritized experience replay [14] and its distributed variants [15] demonstrate that selecting more informative samples can accelerate training. In federated learning, client selection strategies similarly aim to improve efficiency by prioritizing participants based on their contribution.

Our notion of actor importance can be viewed as an extension of these ideas to distributed RL systems. Instead of prioritizing individual samples or clients, we consider the contribution of entire actors. This perspective provides a new dimension for optimizing system behavior, particularly in the context of fault tolerance.

D. Reliability and empirical studies of ML systems

Empirical studies have shown that failures are common in large-scale machine learning workloads and can significantly affect system performance [21]. Testing-based approaches, such as differential testing for RL systems [22], further highlights the complexity of understanding system behavior under varying conditions.

Our work complements these studies by providing a structured analysis of how failures influence both system performance and learning dynamics in distributed RL. By combining empirical evaluation with system-level instrumentation, we bridge the gap between reliability analysis and learning behavior.

VII. Conclusion

In this work, we presented a comprehensive study of fault tolerance in distributed reinforcement learning systems, focusing on the interaction between system behavior and learning dynamics. Through controlled experiments using logical scaling, we analyzed how partial worker failures, recovery mechanisms, and actor contributions influence both training performance and system efficiency.

Our findings show that failures do not uniformly affect learning; their impact ranges from minor disturbance to moderate reward degradation depending on the environment, scaling point, and training configuration. We also demonstrate that checkpointing, while necessary for recovery, is not the dominant contributor to system overhead. Most importantly, we reveal that actor contributions are highly uneven, suggesting that learning is driven by a subset of workers.

These observations lead us to an insight that fault tolerance in distributed RL systems should be learning-aware, prioritizing components that have a greater impact on training outcomes. By aligning system-level decisions with learning behavior, it becomes possible to design more efficient and adaptive training systems.

Overall, this work provides a foundation for rethinking fault tolerance in RL, moving beyond uniform recovery strategies toward approaches that are informed by dynamics of learning itself.

References

[1] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," *IEEE Transactions on Neural Networks*, vol. 9, no. 5, 1998, doi: 10.1109/TNN.1998.712192.

[2] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic Policy Gradient Algorithms," in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014. https://proceedings.mlr.press/v32/silver14.html

[3] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-End Training of Deep Visuomotor Policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016. Available: https://jmlr.org/papers/v17/15-522.html

[4] V. Mnih et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: https://doi.org/10.1038/nature14236.

[5] V. Mnih et al., "Asynchronous Methods for Deep Reinforcement Learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016 https://proceedings.mlr.press/v48/mniha16.html.

[6] L. Espeholt et al., "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. Available: https://proceedings.mlr.press/v80/espeholt18a.html

[7] H. Yu, J. Li, Y. Hua, X. Yuan, and H. Wang, "Cheaper and Faster: Distributed Deep Reinforcement Learning with Serverless Computing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 15, pp. 16539–16547, Mar. 2024, doi: https://doi.org/10.1609/aaai.v38i15.29592.

[8] H. Yu, H. Wang, D. Tiwari, J. Li, and S.-J. Park, "Stellaris: Staleness-Aware Distributed Reinforcement Learning with Serverless Computing," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–17*, Nov. 2024, doi: https://doi.org/10.1109/sc41406.2024.00045.

[9] H. Yu, J. Carter, H. Wang, D. Tiwari, J. Li, and S.-J. Park, "Nitro: Boosting Distributed Reinforcement Learning with Serverless Computing," *Proceedings of the VLDB Endowment*, vol. 18, no. 1, pp. 66–79, Sep. 2024, doi: https://doi.org/10.14778/3696435.3696441.

[10] R. Wei, H. Yu, X. Song, J. Li, D. Tiwari, Y. Mao, and H. Wang, "Multi-Agent Reinforcement Learning with Serverless Computing," in *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25*, Association for Computing Machinery, New York, NY, USA, 225–239. https://doi.org/10.1145/3772052.3772227

[11] H. Zhang, P. Chen, S. Angel, V. Liu, and A. Cardoza, "Fault-tolerant and Transactional Stateful Serverless Workflows," in *Proceedings of USENIX OSDI*, 2020.

[12] G. Candea and A. Fox, "Crash-Only Software," 2005. Available: https://research.cs.wisc.edu/areas/os/ReadingGroup/os-old/Papers/HotOSIX/Candea-CrashOnlySoftware.pdf

[13] E. Jonas et al., "Cloud Programming Simplified: A Berkeley View on Serverless Computing," arXiv:1902.03383, Feb. 2019, Available: https://arxiv.org/abs/1902.03383

[14] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," arXiv:1511.05952, Feb. 25, 2016. https://arxiv.org/abs/1511.05952

[15] D. Horgan et al., "Distributed Prioritized Experience Replay," in *Proceedings of ICLR*, 2018.

[16] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. S. Eugene Ng, and Y. Wang, "GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints," in *Proc. SOSP*, 2023, pp. 364–381. https://doi.org/10.1145/3600006.3613145

[17] Y. Zhong, G. Sheng, J. Liu, J. Yuan, and C. Wu, "Swift: Expedited Failure Recovery for Large-Scale DNN Training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 9, pp. 1644–1656, 2024. https://doi.org/10.1109/TPDS.2024.3429625

[18] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. USENIX OSDI*, 2016. https://dl.acm.org/doi/10.5555/3026877.3026899

[19] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, "Scaling Distributed Machine Learning with the Parameter Server," in *Proc. USENIX OSDI*, 2014.

[20] T. Kim, S. Jeong, J. Lee, S. Lee, and M. Jeon, "Sibylla: To Retry or Not To Retry on Deep Learning Job Failure," in *Proc. USENIX ATC*, 2022. https://www.usenix.org/system/files/atc22-kim-taeyoon.pdf

[21] R. Zhang, W. Xiao, H. Zhang, Y. Liu, H. Lin, and M. Yang, "An empirical study on program failures of deep learning jobs," in *Proc. ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020. https://doi.org/10.1145/3377811.3380362

[22] Q. Pang, Y. Yuan, and S. Wang, "MDPFuzz: Testing models solving Markov decision processes," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2022. https://doi.org/10.1145/3533767.3534388

[23] F. Lai, X. Zhu, H. Madhyastha, M. Chowdhury, "Oort: Efficient Federated Learning via Guided Participant Selection," in *15th USENIX Symposium on Operating Systems Design and Implementation OSDI*, 2021. https://www.usenix.org/conference/osdi21/presentation/lai

[24] C. Smestad and J. Li, "A Systematic Literature Review on Client Selection in Federated Learning," in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (EASE '23)*, 2023. Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/3593434.3593438

[25] L. Wang, Y. Guo, T. Lin, X. Tang, "Delta: Diverse Client Sampling for Fasting Federated Learning," in *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeuIPS '23).*,

2023. Curran Associates Inc., Red Hook, NY, USA, Article 2063, 47626–47668.

[26] Y. Burda, H. Edwards, A. Storkey, O. Klimov, “Exploration by Random Network Distillation,” in ” arXiv:1810.12894, Oct. 30, 2018. https://arxiv.org/abs/1810.12894

[27] B. Eysenbach, A. Gupta, J. Ibarz, S. Levine, “Diversity Is All You Need: Learning Skills Without a Reward Function,” in *International Conference on Learning Representations (ICLR)*, 2019. https://openreview.net/forum?id=SJx63jRqFm

[28] J. Park, I. Hwang, M. W. Lee, H. Oh, M. Lee, Y. Lee, B. Zhang, “On the Importance of Critical Period in Multi-stage Reinforcement Learning,” in ” arXiv:2208.04832, Aug. 09, 2022. https://arxiv.org/abs/2208.04832

[29] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation (NSDI’12)*, 2012. USENIX Association, USA, 2.

[30] P. Carbone, S. Ewen, S. Haridi, A. Katsifodimos, V. Markl, K. Tzoumas, “Apache Flink™: Stream and Batch Processing in a Single Engine,” in *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering, 36(4), 28-38*,

[31] J. Dean and S. Ghemawat, “MapReduce: simplified data processing on large clusters,” in *BCommun. ACM 51, 1 (January 2008), 107–113. https://doi.org/10.1145/1327452.1327492*,

[32] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the wild: characterizing and optimizing the serverless workload at a large cloud provider,” in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC’20)*, 2020. USENIX Association, USA, Article 14, 205–218.